

DD: NOT-01 — Notification Service

Developer implementation guide: DD_NOT-01-Notification_Service-DEV_IMPL_GUIDE-v1.3.md.

1. Purpose

This rewritten DD is the developer-facing version of the module contract DD_NOT-01-Notification_Service-v1.3.md.

Verdict: ■ New | **Wave:** 1 | **Group:** Cross-cutting Foundation Services | **Version:** v1.3 Cross-cutting service consumed by many DDs. Owns three concerns: (1) **artifact rendering** — producing rendered output (PDFs, email bodies, SMS texts, future formats) from templates with placeholder substitution; (2) **channel delivery** — dispatching notifications through pluggable channel adapters (initial scope: **email and SMS**; architecture extensible to add WhatsApp, Telegram, push, or any other channel by adding a new adapter); (3) **routing and preferences** — deciding which event triggers which channels using which template, respecting per-customer preferences and per-operator regulatory rules. **Version history:** see CHANGELOG_NOT-01-Notification_Service.md in the same directory.

The goal of this rewrite is to make the DD easier for a mid-level developer to implement without losing the detailed source contract.

2. Implementation Scope

This DD should be implemented as part of the owning SOPHIX capability, not as an isolated service unless the deployment architecture explicitly separates that capability.

Implement this DD with these rules:

- Use the owning module APIs/repositories for authoritative writes.
- Use approved internal APIs/client wrappers when reading another module's data.
- Do not query another module's database tables directly from business flow code.
- Treat worker names as logical Camunda external-task topics, not separate deployments.
- One worker application may handle multiple topics, but metrics, retries, and logs must remain visible per topic.
- Emit success events only after the authoritative state commit succeeds.
- Use outbox publishing for Kafka events.

3. Runtime Metadata

Item	Value
Source file	/Users/macbook/Downloads/Sophix_V3_BSS_DDs_MD_2026-05-27/10_other/DD_NOT-01-Notification_Service-v1.3.md
Version	v1.3
DD type	module contract
BPMN/process keys	Not explicitly defined
Drools packages	Not explicitly defined

4. Runtime Contracts To Implement

4.1 APIs

- GET /api/admin/notifications/dashboard
- GET /api/notifications/preferences
- POST /api/admin/notifications/ntf_01HZ.../resend
- POST /api/admin/notifications/send
- POST /api/admin/notifications/{id}/resend
- POST /api/admin/templates

- PUT /api/notifications/preferences

4.2 Tables

- No CREATE TABLE block is explicitly declared in this DD.

For each table in this DD, implement the schema with:

- a short table comment explaining what the table is for;
- column comments or migration notes explaining each field;
- constraints and indexes from the DD;
- seed data where the DD provides operator-specific sample rows.

4.3 Worker Topics

- No worker topics are explicitly declared in this DD.

Worker-topic guidance:

- A BPMN service task uses the topic.
- A worker application subscribes to the topic.
- The topic handler validates input variables, calls the owning module/API, writes outputs back to Camunda, and records metrics.
- Do not treat the topic string as a shell command or Kubernetes deployment name.

4.4 Events

- InvoiceIssued
- InvoiceOverdue
- InvoiceStatusChanged
- PaymentApplied
- PaymentFailed

Event guidance:

- Write events through the transactional outbox.
- Include operation id, aggregate id, operator code, actor, and correlation data.
- Consumers should be able to rebuild downstream state without querying workflow internals.

5. Developer Implementation Checklist

1. Add or verify database migrations and seed rows.
2. Add or verify config rows for the operator/module.
3. Implement request/command DTOs and validation.
4. Implement idempotency and in-flight operation checks where the DD requires them.
5. Implement Drools fact mapping and package deployment where rules are used.
6. Implement BPMN process, service tasks, gateways, timers, and message catches where the DD is a flow.
7. Implement worker-topic handlers using owning module APIs.
8. Implement compensation paths and manual recovery paths.
9. Emit success/rejected events through the outbox.
10. Add smoke tests for happy path, validation failure, dependency failure, and retry/idempotency.

6. Infrastructure And AWS Notes

Deploy this DD with the existing SOPHIX platform components unless the owning module requires isolation:

Concern	Recommendation
API/runtime	Run inside the owning module deployment or existing workflow API pods.
Workers	Consolidate low-volume topics into shared worker apps; keep per-topic metrics.
Database	Use the owning module PostgreSQL schema and migration pipeline.
Kafka	Use the foundation Kafka/MSK setup and transactional outbox.
Camunda	Deploy BPMN files through the workflow deployment pipeline.
Drools	Deploy KJAR/rule artifacts through the approved rules pipeline.
Secrets	Use the platform secret manager; on AWS prefer Secrets Manager/Parameter Store with IRSA.
Observability	Alert on stuck operations, failed workers, outbox backlog, and external dependency failures.

7. Original DD Detail Preserved

The following section preserves the detailed source contract for implementation and review.

Verdict: ■ New | **Wave:** 1 | **Group:** Cross-cutting Foundation Services | **Version:** v1.3

Cross-cutting service consumed by many DDs. Owns three concerns: (1) **artifact rendering** — producing rendered output (PDFs, email bodies, SMS texts, future formats) from templates with placeholder substitution; (2) **channel delivery** — dispatching notifications through pluggable channel adapters (initial scope: **email and SMS**; architecture extensible to add WhatsApp, Telegram, push, or any other channel by adding a new adapter); (3) **routing and preferences** — deciding which event triggers which channels using which template, respecting per-customer preferences and per-operator regulatory rules.

Version history: see `CHANGELOG_NOT-01-Notification_Service.md` in the same directory.

Glossary

Term	Meaning
Template	Text with placeholders (variables filled at render time). Templates have a format indicating what they produce: PDF, EMAIL_HTML, EMAIL_TEXT, EMAIL_SUBJECT, SMS_TEXT, and future formats as needed. Any templating engine that supports placeholder substitution can be used (Handlebars, Thymeleaf, Mustache, plain <code>\${var}</code> substitution, HTML-to-PDF with placeholders, etc.) — the engine is a deployment choice, not a DD constraint.
Rendering	Filling a template's placeholders with values from an event payload (and supporting data: customer snapshot, operator branding, locale resources). Produces a rendered artifact: a PDF byte-stream, an HTML email body string, an SMS text string.
Artifact	The output of rendering. PDFs are stored in MinIO (per <code>FOUNDATION_FILE_STORAGE.md</code>) and referenced by URL; email/SMS rendered text is held in-memory and passed straight to the channel adapter.
Channel	A push-delivery medium: initial scope is email and SMS . The channel adapter contract is designed to be extensible — adding WhatsApp, Telegram, in-app push, voice, or any other channel later is implementing a new adapter + adding its template format, with no changes to the routing or preference model.
Channel adapter	A pluggable implementation that knows how to send through one specific channel — talks to SMTP for email, the SMSC API for SMS, and any future channel's API. One adapter per channel; per-operator configuration.
Routing rule	A per-operator mapping: when event X happens, send through channels [A, B, ...] using templates [T1, T2, ...], conditional on customer preferences and event payload.
Notification preference	A customer's stated preference about how to be contacted: which channels are opted in, which are opted out, preferred locale, preferred time-of-day for non-urgent messages. Owned by NOT-01 (in <code>customer_notification_preference</code> table).
Delivery attempt	One try at sending a message through a channel. Captured in <code>notification_delivery_attempt</code> . Multiple attempts per notification if retries are needed.
Bounce / failure	A channel-reported failure: email bounce, SMS delivery failure. NOT-01 categorizes (transient vs permanent) and decides whether to retry, fall back to another channel, or escalate.

A — Target

1. What this process is about

NOT-01 is the service that turns events from across the platform into something a customer or admin actually sees — a PDF stored for later download, an email in their inbox, an SMS on their phone. It has three internal concerns that work together:

Concern A — Artifact rendering. When an event arrives, NOT-01 looks up the templates configured for that event + operator + locale + channel, fills the placeholders with values from the event payload (plus supporting data like the customer snapshot and operator branding), and produces the rendered output. PDFs (invoices, credit/debit notes, tax invoices, dunning letters) are stored in MinIO per `FOUNDATION_FILE_STORAGE.md` and the URL is written back to the source entity's metadata. Email bodies and SMS texts are rendered in-memory and passed straight to the channel adapter.

Templates are just text with placeholders. The templating engine is a deployment choice — Handlebars, Mustache, Thymeleaf, HTML-to-PDF tools, or plain string substitution all work. NOT-01 doesn't mandate one engine; it provides a `TemplateEngine` interface and selects the implementation based on the template's format (PDF templates use a PDF-capable engine; email and SMS use a text/HTML engine).

Concern B — Channel delivery. When events require pushing a message to a customer or admin, NOT-01 dispatches through configured channels. **Initial scope: email and SMS.** The architecture uses pluggable channel adapters — adding WhatsApp, Telegram, in-app push, voice, or any other channel later is implementing a new adapter (and adding its template format) without changing routing, preferences, audit, or anything else in NOT-01.

Concern C — Routing and preferences. "Which channels do I use, which templates, in what order, respecting which preferences" is configuration, not code. Per-operator routing rules map event types to channel + template lists. Per-customer notification preferences override defaults (customer opted out of SMS but accepts email). Per-operator regulatory rules enforce constraints (no SMS after 21:00 in some markets; transactional vs marketing distinction).

How rendering and channel delivery relate

For an event like `InvoiceIssued`, NOT-01 typically does both:

1. **Renders the PDF first** (using the operator's PDF template) — produces the artifact, stores it in MinIO, gets a URL
2. **Renders channel artifacts** (email subject + body, SMS text) using their respective templates
3. **Dispatches through channels** — sends email with the PDF attached or linked, sends SMS with a link to the PDF (the URL points to `BIL-02-READ-01`'s PDF endpoint, which proxies the stored artifact)

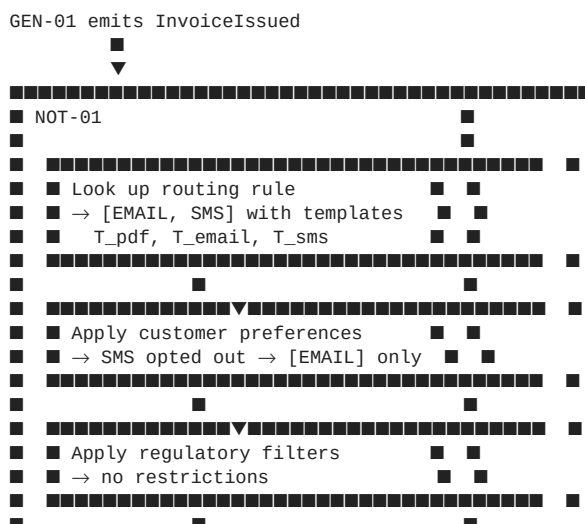
For an event like `PtpRegistered` that doesn't need a PDF:

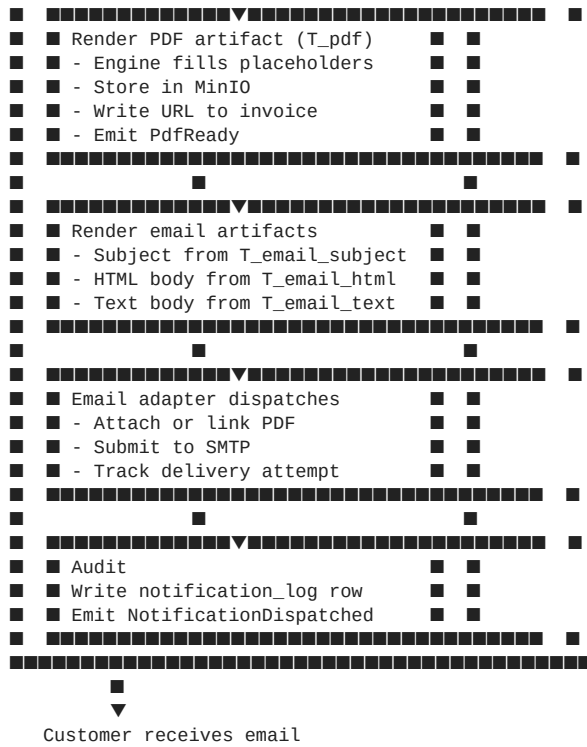
1. PDF rendering is skipped
2. NOT-01 renders the email/SMS templates and dispatches a confirmation ("We've registered your promise to pay 6,402.50 KES by 25 May")

For an admin action like `PdfRerenderRequested` (admin asked `READ-01` to retry a failed render):

1. Only PDF rendering runs — produces a new PDF, replaces the URL in `invoice.metadata.pdf_url`
2. Channel dispatch may or may not run depending on whether a re-notification is requested

Flow overview for the `InvoiceIssued` case





Initial channel scope

Channel	Use cases	Adapter talks to	Special considerations
Email	Invoice notifications, dunning letters, confirmations, OTPs, marketing	Operator's SMTP server or transactional email service	HTML body + plain text fallback; PDF attachments supported; bounces tracked via SMTP feedback
SMS	Quick notifications, OTPs, urgent alerts, short reminders	Operator's SMSC API or aggregator	Strict length limits (typically 160 chars; longer concatenated as multi-part); per-country sender ID rules; per-operator time-of-day restrictions; cost per send; delivery receipts (DLR) tracked when supported

Extensibility — adding a channel later

Adding a channel (WhatsApp, Telegram, in-app push, voice, paper mail, anything else) is a contained change:

1. Implement a new ChannelAdapter for that channel
2. Define template format(s) for that channel (e.g., WHATSAPP_TEMPLATE_REF for pre-approved WhatsApp templates; PUSH_PAYLOAD for push)
3. Add per-operator channel config row (credentials, identifiers)
4. Operators add routing rules that include the new channel
5. Customer preference table gets a new opt-in field

The routing engine, the preferences engine, the failure handling, the audit, the admin operations all remain unchanged. The DD's design separates "what the channel is" from "how routing and preferences work" precisely to make this extension cheap.

Channels NOT in initial scope: WhatsApp, Telegram, in-app push, voice/IVR, paper mail, in-app messages (notification feed within the portal/app — those are owned by the consuming UI, not NOT-01).

What NOT-01 explicitly does not own

- **Event sources** — invoice issuance, payment events, subscription lifecycle, etc. are owned by their respective DDs; NOT-01 just consumes events
- **Customer master** — CRM owns customer identity, contact details (email, phone), language preference at the customer level. NOT-01 reads these via a service call when composing notifications
- **Dunning policy** — BIL-04 Collections decides "step 1 reminder at D+3, step 2 at D+7, etc." and emits events to NOT-01; NOT-01 doesn't decide the schedule

- **Marketing campaigns** — separate marketing system owns campaign creation, segmentation, scheduling. May piggyback on NOT-01's channel adapters for actual sending, but the campaign logic is elsewhere
- **In-app message rendering** — owned by the UI (portal, mobile app)
- **Voice / IVR notifications** — out of scope for initial release; if needed later, added as a new channel adapter
- **Long-running customer support conversations** — out of scope; NOT-01 handles transactional/notification messages, not bidirectional conversations
- **Authentication tokens / sessions** — owned by FOUNDATION_AUTH.md; NOT-01 uses these to authenticate to channel providers but doesn't own them
- **Templating engine choice** — deployment decision; NOT-01 specifies the contract, not the implementation

2. Business rules

Rules grouped by phase: D (artifact rendering), R (routing), C (channel delivery), F (failure handling), P (preferences), A (audit), O (admin operations).

Rule group D — Artifact rendering

ID	Rule	Triggered on
R-NOT-01-D-1	NOT-01 consumes events that require rendered artifacts: InvoiceIssued, CreditNoteIssued, DebitNoteIssued, TaxInvoiceSigned, DunningStepTriggered (from BIL-04 Collections), PdfRerenderRequested (admin-triggered from READ-01), and any event whose routing rule includes channels that need rendered output (every channel needs some kind of template-rendered text or document).	Events requiring rendering
R-NOT-01-D-2	NOT-01 looks up the templates configured for the event. Each routing rule references one or more templates by ID. Templates are identified by (operator_code, template_format, template_purpose_code, locale, version) where template_format is PDF, EMAIL_HTML, EMAIL_TEXT, EMAIL_SUBJECT, SMS_TEXT, or any future format. If a needed template doesn't exist, the missing template is logged to render_failure_queue and a RenderFailed event is emitted.	Template lookup
R-NOT-01-D-3	Templates are text with placeholders. The templating engine is a deployment choice; NOT-01 provides a TemplateEngine interface and the deployment registers implementations per template_format. Default implementations may use Handlebars or similar for text formats and an HTML-to-PDF engine for the PDF format — but the DD does NOT mandate specific libraries.	Engine selection
R-NOT-01-D-4	Rendering inputs: the event payload + customer snapshot (fetched from the source entity, which captured it at issuance) + operator branding assets + locale resource bundle. Output: the rendered artifact — a byte stream for PDF, a string for email/SMS texts.	Rendering inputs
R-NOT-01-D-5	PDF artifacts are stored in MinIO per FOUNDATION_FILE_STORAGE.md conventions: bucket invoices (for billing documents), bucket dunning (for dunning letters), key pattern {document_type_prefix}/{year}/{month}/{document_id}.pdf. The MinIO key is written back to the source entity (e.g., invoice.metadata.pdf_url = "invoices/2026/05/inv_01HZ...pdf"). Email and SMS artifacts are NOT stored — they're held in memory and passed straight to the channel adapter.	Artifact storage
R-NOT-01-D-6	On successful PDF render and storage, NOT-01 emits PdfReady with entity_id, entity_type, pdf_url, rendered_at. Consumed by READ-01 for cache invalidation, by analytics, and (if part of an InvoiceIssued flow) by NOT-01's own dispatch step which has been waiting for the PDF before sending.	PDF success emission

ID	Rule	Triggered on
R-NOT-01-D-7	If rendering fails (template missing, malformed data, engine error), NOT-01 retries with exponential backoff (5min, 15min, 30min, 1h, 2h, 4h, 8h, 24h). After 8 transient retries, the entry moves to <code>render_failure_queue</code> for admin review and emits <code>RenderFailed</code> .	Failure retry
R-NOT-01-D-8	Admin can trigger PDF re-render via <code>PdfRenderRequested</code> event (from READ-01 admin operation) or directly via NOT-01's admin REST. On re-render, the new PDF replaces the old in <code>MinIO</code> (versioned per <code>FOUNDATION_FILE_STORAGE.md</code>) and the source entity's metadata <code>.pdf_url</code> is updated. Re-render emits <code>PdfReady</code> again.	Re-render
R-NOT-01-D-9	Rendering is per-operator and per-template-purpose. Some templates are optional per operator: pro forma PDFs may be disabled (<code>operator_config.render_pro_forma = false</code>); some routing rules may skip channels that don't apply. Missing optional templates are NOT failures.	Operator-configurable rendering

Rule group R — Routing

ID	Rule	Triggered on
R-NOT-01-R-1	A routing rule defines: <code>(operator_code, event_type) → list of {channel, template_refs, conditions, priority}</code> entries. <code>template_refs</code> is the list of template IDs needed for that channel (e.g., email needs <code>EMAIL_SUBJECT</code> , <code>EMAIL_HTML</code> , <code>EMAIL_TEXT</code> ; SMS needs <code>SMS_TEXT</code> ; channels that need an attached/linked PDF reference a PDF template). Stored in <code>notification_routing_rule</code> .	Routing decision
R-NOT-01-R-2	Routing rules are operator-configurable. Operators commit their own rule sets per environment. NOT-01 ships a default rule set per operator for common events (invoice issued, payment received, dispute acknowledgement, etc.); operators customize.	Per-operator config
R-NOT-01-R-3	After resolving routing rule list, NOT-01 applies the customer's notification preferences (rule group P). Channels the customer opted out of are removed from the list. If all channels are removed, NOT-01 logs the event as suppressed — emits <code>NotificationSuppressed</code> for audit; does not send.	Preferences filter
R-NOT-01-R-4	After preferences, NOT-01 applies regulatory and operator policy filters. E.g., if the event triggers an SMS but it's currently 22:30 local time and the operator's regulatory config forbids SMS after 21:00, the SMS is queued for delivery at the next allowed time.	Regulatory filter
R-NOT-01-R-5	The final channel list is the input to dispatch. For each channel, NOT-01 renders the channel's required templates (per rule group D) and queues a delivery with the rendered artifact(s), the customer's contact details, and the PDF URL if a PDF is referenced.	Hand-off to dispatch
R-NOT-01-R-6	Some events are explicitly NOT routed to any channel (purely internal, for audit only). E.g., <code>InvoiceStatusChanged</code> is consumed by READ-01 for cache invalidation but should not produce customer-facing notifications — the routing rule list is empty for that event.	Internal-only events

Rule group C — Channel delivery

ID	Rule	Triggered on
R-NOT-01-C-1	Each channel has a pluggable adapter implementing the ChannelAdapter interface. Methods: prepare(rendered_artifacts, recipient_address, attachments) returns a channel-specific message representation; send(message) returns a DeliveryResult. The contract is channel-agnostic; adapters handle channel-specific mechanics. Initial scope: EmailAdapter, SmsAdapter. Future adapters (WhatsApp, Telegram, push, voice, paper mail) implement the same interface.	Adapter contract
R-NOT-01-C-2	Per-operator channel configuration in channel_operator_config: which adapter implementation to use, what credentials, what sender identifiers (email sender address, SMS sender ID / shortcode). Sensitive values stored via the standard secret management per FOUNDATION_AUTH.md.	Operator config
R-NOT-01-C-3	Email specifics: HTML body + plain text fallback rendered from EMAIL_HTML and EMAIL_TEXT templates respectively; subject from EMAIL_SUBJECT. PDF attachments referenced by MinIO key fetched at send time (operator config decides attach-vs-link). Recipient address from customer master via CRM. Sender from operator's verified sender configuration. Bounces tracked via SMTP feedback / inbound bounce mailbox parsing.	Email specifics
R-NOT-01-C-4	SMS specifics: message text rendered from SMS_TEXT template, truncated to channel limit (typically 160 chars for single-SMS; longer messages are concatenated multi-part SMS at additional cost per part). Recipient phone number from customer master, validated against operator's allowed regions. Delivery receipts (DLR) tracked when the channel supports them.	SMS specifics
R-NOT-01-C-5	Each successful send writes a notification_delivery_attempt row with channel, recipient, template_refs, status, sent_at, channel-specific response data. Each failure writes the same with status and error category.	Delivery audit
R-NOT-01-C-6	Adding a new channel later is contained: implement a new ChannelAdapter, define template format(s) for that channel, add per-operator channel_operator_config row, add routing rules. The routing engine, preferences engine, failure handling, audit, and admin operations remain unchanged.	Extensibility

Rule group F — Failure handling

ID	Rule	Triggered on
R-NOT-01-F-1	Failures are categorized: TRANSIENT (gateway timeout, rate limit, temporary network issue — retry sensible), PERMANENT_RECIPIENT (invalid email/phone — won't succeed without contact update), PERMANENT_TEMPLATE (template rejection by channel — needs admin fix), PERMANENT_BUSINESS_RULE (recipient opted out at carrier level, blocked region).	Failure categorization
R-NOT-01-F-2	TRANSIENT failures are retried with exponential backoff: 1min, 5min, 15min, 30min, 1h, 2h, 4h. After 7 transient retries (~8 hours), the notification is escalated to EscalatedFailure — emits an event for ops monitoring and is no longer auto-retried.	Transient retry
R-NOT-01-F-3	PERMANENT_RECIPIENT failures trigger fallback: NOT-01 attempts the next channel in the routing rule's priority order. If all channels fail with PERMANENT_RECIPIENT, the notification is marked UNDELIVERABLE and an event is emitted so the operator can attempt to re-engage the customer (admin contact-details update workflow).	Fallback to next channel

ID	Rule	Triggered on
R-NOT-01-F-4	PERMANENT_TEMPLATE failures (template rejected by the channel provider — e.g., content policy violation, format error) immediately escalate to ops; the offending template is marked DISABLED in template so subsequent notifications skip it and emit a routing warning.	Template rejection
R-NOT-01-F-5	Email bounce processing: bounces (received via SMTP bounce handler) update customer_notification_preference to mark email as BOUNCED (soft) or INVALID (hard); INVALID marks the email channel as not-usable until admin updates the customer's email.	Email bounce
R-NOT-01-F-6	Each failure writes to notification_delivery_attempt with category and error detail. Repeated failures on the same notification create multiple rows (one per attempt), preserving the full retry history.	Failure audit

Rule group P — Preferences

ID	Rule	Triggered on
R-NOT-01-P-1	Customer notification preferences are stored in customer_notification_preference: per-customer, per-channel opt-in/opt-out, preferred locale, preferred time-window. Defaults come from operator config; customer overrides.	Preference storage
R-NOT-01-P-2	Customers can update preferences via the self-service portal (REST endpoints owned by NOT-01) or via channel-specific opt-out flows (e.g., reply "STOP" to SMS triggers an SMS opt-out per regulations).	Customer-initiated update
R-NOT-01-P-3	Preferences distinguish between transactional (legally required: invoices, payment confirmations, security alerts — customers cannot opt out) and marketing/informational (customers can opt out). Routing rules specify which category each event falls under; opt-out applies only to marketing/informational.	Transactional vs marketing
R-NOT-01-P-4	For marketing/informational notifications, NOT-01 must use current customer_account_notification_pref and may reference ILM's customer_consent_ledger for audit/provenance. NOT-01 does not own consent capture. ILM owns the append-only consent ledger and current account preferences.	Consent ownership
R-NOT-01-P-4	Locale preference governs which template variant is selected. If no template exists in the preferred locale, fall back to operator's default locale; if no default-locale template exists either, fall back to English; if no English template, log failure.	Locale fallback
R-NOT-01-P-5	Time-window preference (customer wants notifications between 09:00-18:00 only): non-urgent notifications outside the window are queued for delivery at the next start of window. Urgent events (security alerts, payment failure) bypass the window. Urgency is set in the routing rule.	Time-window respect

Rule group A — Audit

ID	Rule	Triggered on
R-NOT-01-A-1	Every notification dispatch attempt creates a notification_log aggregate row with notification_id (ULID), event_type, customer_id, operator_code, channels_attempted, final_status, dispatched_at. Linked to one or more notification_delivery_attempt rows.	Aggregate audit
R-NOT-01-A-2	Audit retention: 10 years per operator default (matches invoice retention since invoice notifications are part of the audit trail of billing communications). Operator can override.	Retention

ID	Rule	Triggered on
R-NOT-01-A-3	Audit queries support: by customer (all notifications to a customer in a period), by event (which notifications were sent for a specific InvoiceIssued event), by channel (delivery rates per channel per operator per period), by failure category.	Query patterns
R-NOT-01-A-4	Audit events emitted: NotificationDispatched, NotificationFailed, NotificationSuppressed, NotificationEscalated, BounceProcessed. Consumed by analytics for delivery metrics dashboards.	Event emission

Rule group O — Admin operations

ID	Rule	Triggered on
R-NOT-01-O-1	Admin can manually resend a notification: POST /api/admin/notifications/{id}/resend. Re-runs Module 2 (and Module 1 if a fresh PDF is requested) for the original event payload. Captured in audit with manual_resend_by user ID.	Manual resend
R-NOT-01-O-2	Admin can manually send an ad-hoc notification (not driven by an event): POST /api/admin/notifications/send with event_type override, customer_id, channels, message content. Used for support scenarios. Requires NOTIFICATION_SENDER role.	Manual send
R-NOT-01-O-3	Admin template management UI: upload new template versions, preview rendering with sample data, activate / deactivate templates. Templates are versioned; previous versions remain in the database for audit. Per-channel approval workflows (where the channel requires pre-approval of templates by a third party, e.g., when adding WhatsApp later) are owned by the relevant channel adapter's admin flow.	Template management
R-NOT-01-O-4	Admin delivery dashboard: real-time metrics (notifications per minute per channel), failure rates, queue depths, top failure reasons, customers with delivery issues. Operational tool for the notifications team.	Dashboard
R-NOT-01-O-5	Admin failure queue: lists notifications in EscalatedFailure state for human review. Admin can retry, edit content, change channel, or abandon.	Failure recovery
R-NOT-01-O-6	Admin can pause/resume routing for an event type per operator (e.g., during a downstream incident, pause InvoiceIssued SMS notifications to avoid customer confusion). Captured in audit.	Emergency routing controls

3. Module owner and impacted modules

Role	Module	What it does
Owner	New notification module on its own cluster (let's call it cluster N — separate from billing cluster C to allow independent scaling)	Owns rendering, channel delivery, routing, preferences, audit. Houses the three internal modules.
Object storage	MinIO per FOUNDATION_FILE_STORAGE.md	Stores rendered PDFs (invoices, dunning buckets)
Channel adapter dependencies	Per-operator external services (SMTP server / transactional email service for email; SMSC API / aggregator for SMS)	NOT-01 talks to these via configured adapters; future channels add their own external dependencies
Upstream events consumed	Many: InvoiceIssued, CreditNoteIssued, DebitNoteIssued, TaxInvoiceSigned, PaymentApplied, PtpRegistered, PtpBroken, DisputeOpened, DisputeResolved*, InvoiceOverdue, DunningStepTriggered, PdfRerenderRequested, subscription events (to be defined), KYC events, etc.	Drive rendering and dispatch decisions

Role	Module	What it does
Downstream events emitted	PdfReady, PdfRenderFailed, NotificationDispatched, NotificationFailed, NotificationSuppressed, NotificationEscalated, BounceProcessed	Consumed by READ-01 (cache invalidation), source DDs (status updates), analytics
Frontend	Admin console — template management UI, delivery dashboard, failure recovery queue; Customer portal — notification preferences page	UI teams own the frontends; NOT-01 provides the backend
Engines	Templating engine for placeholder substitution — deployment chooses (Handlebars, Mustache, Thymeleaf, HTML-to-PDF tools, etc.). No Drools, no Camunda — NOT-01 is event-driven processing with retry/fallback logic in code	—

4. Foundation references

DB → FOUNDATION_DB.md · Kafka → FOUNDATION_KAFKA.md · Cache → FOUNDATION_CACHE.md (Redis used for idempotency keys and throttling) · Auth → FOUNDATION_AUTH.md (introduces NOTIFICATION_SENDER, TEMPLATE_MANAGER roles) · File Storage → FOUNDATION_FILE_STORAGE.md (MinIO buckets for rendered PDFs).

Technical design

Module placement

New notification module deployed on its own cluster (cluster N). Cluster choice rationale: notification traffic has different scaling characteristics from billing (spiky around cycle close + dunning waves) and different external dependencies (SMTP, SMS gateways, etc.) that benefit from network isolation. Cluster N has independent CPU/memory/network capacity from cluster C.

Adds the following new tables: template, notification_routing_rule, channel_operator_config, customer_notification_preference, notification_log, notification_delivery_attempt, render_failure_queue. All on cluster N. (When new channels are added later, they may need additional tables — e.g., a token store for push, a chat-link store for messaging-app channels — added at that time.)

Code structure

```

modules/notification/
src/main/java/com/sophix/notification/
  rendering/                                ← Artifact rendering
  service/
    ArtifactRenderingService.java           ← orchestrates template resolution → render → store/return
    TemplateLookupService.java             ← resolves (operator, format, purpose_code, locale) → template
    PdfStorageService.java                 ← writes PDFs to MinIO per FOUNDATION_FILE_STORAGE
  engine/
    TemplateEngine.java                   ← interface
    TemplateEngineRegistry.java           ← registers per-format engine implementations at startup
    # Concrete engine implementations      are deployment choices (Handlebars, Mustache, HTML-to-PDF, etc.)
  kafka/
    RenderEventConsumer.java              ← consumes events that need rendering
  domain/
    Template.java
    TemplateFormat.java                   ← enum: PDF, EMAIL_HTML, EMAIL_TEXT, EMAIL_SUBJECT, SMS_TEXT, ...
    RenderResult.java
  routing/                                  ← Routing & Preferences
  service/
    RoutingResolverService.java           ← event → channels + template purpose codes
    PreferenceFilterService.java          ← apply customer preferences
    RegulatoryFilterService.java          ← apply operator regulatory rules
  domain/
    NotificationRoutingRule.java
    CustomerNotificationPreference.java
    RoutingDecision.java
  dispatch/                                ← Channel Delivery
  service/
    DispatchService.java                  ← entry point – receives RoutingDecision, dispatches
    ChannelDispatcher.java                ← per-channel queue handler
  adapter/
    ChannelAdapter.java                   ← interface – channel-agnostic contract
    EmailAdapter.java                     ← SMTP / transactional email service (initial scope)
    SmsAdapter.java                       ← SMSC / aggregator (initial scope)

```

```

    # Future adapters (WhatsApp, Telegram, push, voice, paper mail) add new files here
domain/
    DeliveryAttempt.java
    DeliveryResult.java
    FailureCategory.java
retry/
    service/
        RetryScheduler.java
rest/
    AdminNotificationResource.java
    AdminTemplateResource.java
    CustomerPreferenceResource.java
repository/
    TemplateRepository.java
    RoutingRuleRepository.java
    CustomerPreferenceRepository.java
    NotificationLogRepository.java
    DeliveryAttemptRepository.java
    RenderFailureQueueRepository.java
src/main/resources/
    config/liquibase/changelog/notification/
        010_template.xml
        020_notification_routing_rule.xml
        021_channel_operator_config.xml
        030_customer_notification_preference.xml
        040_notification_log.xml
        041_notification_delivery_attempt.xml
        042_render_failure_queue.xml

```

End-to-end flow — InvoiceIssued

Step 1 – Event consumed

RenderEventConsumer reads InvoiceIssued from billing topic
 Extracts: invoice_id, customer_id, operator_code, invoice_type_code, payload

Step 2 – Idempotency check (Redis)

Key: notification:processed:{event_id}
 If exists → skip (already processed); ack event and return
 If not → set key with 24h TTL, continue

Step 3 – Route resolution

- RoutingResolverService.resolve("InvoiceIssued", operator_code, payload)
 - [{channel: EMAIL, priority: 1, urgency: NORMAL, purpose: INVOICE_CYCLE_POSTPAID}, {channel: SMS, priority: 2, urgency: NORMAL, purpose: INVOICE_CYCLE_POSTPAID}]
- PreferenceFilterService.apply(customer_id, channels)
 - customer opted out of SMS marketing → SMS is transactional so kept; if it were marketing, dropped
- RegulatoryFilterService.apply(operator_code, channels, now)
 - no restrictions → list unchanged

Step 4 – Render PDF (if any channel needs it)

- TemplateLookupService.resolve(operator_code, PDF, "INVOICE_CYCLE_POSTPAID", customer.locale)
 - Template row, OR null
- If null → render_failure_queue + RenderFailed event + skip PDF-dependent channels
- TemplateEngineRegistry.engineFor(PDF).render(template, payload, customer_snapshot)
 - byte[] PDF
- PdfStorageService.store(operator, "invoices/2026/05/inv_01HZ...pdf", bytes)
 - MinIO key
- Update invoice.metadata.pdf_url via the billing API (or direct DB write if same cluster – TBD)
- Emit PdfReady

Step 5 – Render channel artifacts per channel and dispatch

For each remaining channel:
 For each template format the channel needs (e.g., EMAIL needs SUBJECT + HTML + TEXT):
 Resolve template + render
 DispatchService.queue(channel, RoutingDecision, rendered_artifacts, pdf_url_if_applicable)
 → internal Kafka topic: notification.dispatch.{channel}

EmailAdapter consumer:

- Resolve customer's email from CRM
- Compose with rendered subject + HTML + text + PDF attachment (or link)
- Send via SMTP
- On 2xx: notification_delivery_attempt row with status=SENT
- On 4xx/5xx: notification_delivery_attempt row with status=FAILED + category
- If TRANSIENT failure: retry scheduler queues retry
- If PERMANENT_RECIPIENT: fallback to next channel in priority order

SmsAdapter consumer (in parallel):

- Resolve customer's phone from CRM
- Send rendered SMS text via SMSC API (truncate or split if needed)
- Track DLR if supported

Step 6 – Audit and downstream

- notification_log aggregate row written (final status across all channels)
- Emit NotificationDispatched (or NotificationFailed if all channels failed)

Retry flow

```

RetryScheduler runs every 1 minute
Queries notification_delivery_attempt WHERE status = 'PENDING_RETRY' AND next_attempt_at <= NOW()
For each:
  BEGIN TRANSACTION
  SELECT FOR UPDATE
  If still PENDING_RETRY:
    DispatchService.retry(attempt)
    Update attempt: increment attempt_count, set next_attempt_at based on backoff schedule
  After max attempts: set status = ESCALATED, emit NotificationEscalated
  COMMIT

```

Data model

template

Unified table for all template formats (PDF, email parts, SMS text, and future formats). Replaces the separate document_template and message_template tables a multi-table design would have used — templates are all "text with placeholders" regardless of what they produce.

Column	Type	Notes
id	TEXT PK	ULID
operator_code	TEXT NOT NULL	
template_format	TEXT NOT NULL	PDF / EMAIL_HTML / EMAIL_TEXT / EMAIL_SUBJECT / SMS_TEXT / future formats
template_purpose_code	TEXT NOT NULL	E.g., INVOICE_CYCLE_POSTPAID, CREDIT_NOTE, TAX_INVOICE, DUNNING_STEP_1, PTP_CONFIRMATION, DISPUTE_ACKNOWLEDGMENT. Operator catalog.
locale	TEXT NOT NULL	E.g., en, fr, sw
version	INTEGER NOT NULL	Templates are versioned; current version is the highest active
status	TEXT NOT NULL	ACTIVE / DRAFT / ARCHIVED / DISABLED
engine_type	TEXT NOT NULL	Hint to the rendering layer about which engine to use (HANDLEBARS, MUSTACHE, THYMELEAF, HTML_TO_PDF, etc.) — deployment-specific
template_payload	BYTEA NOT NULL	The actual template text/file (HTML, Handlebars source, Mustache source, etc.)
placeholder_schema	JSONB NULL	Declared list of placeholders the template expects, with types — used for admin validation when uploading
sample_data	JSONB NULL	Preview data for admin testing
created_by	TEXT NOT NULL	Admin user ID
created_at	TIMESTAMPTZ NOT NULL	

Indexes: (operator_code, template_format, template_purpose_code, locale, status, version DESC) for lookup; UNIQUE on (operator_code, template_format, template_purpose_code, locale, version).

Sample data (illustrative, partial):

operator	template_format	template_purpose_code	locale	engine_type
WIK	PDF	INVOICE_CYCLE_POSTPAID	en	HTML_TO_PDF
WIK	EMAIL_SUBJECT	INVOICE_CYCLE_POSTPAID	en	HANDLEBARS
WIK	EMAIL_HTML	INVOICE_CYCLE_POSTPAID	en	HANDLEBARS
WIK	EMAIL_TEXT	INVOICE_CYCLE_POSTPAID	en	HANDLEBARS
WIK	SMS_TEXT	INVOICE_CYCLE_POSTPAID	en	HANDLEBARS
WIK	EMAIL_HTML	INVOICE_CYCLE_POSTPAID	sw	HANDLEBARS
WIK	SMS_TEXT	PTP_CONFIRMATION	en	HANDLEBARS

How to read: For a Wananchi Kenya cycle invoice in English, NOT-01 finds five templates — one PDF, three email parts (subject + HTML body + text body), one SMS text. For a PTP confirmation, only an SMS text template exists in this example

— meaning PTP confirmations are SMS-only for this operator's English locale (the routing rule only configures SMS for that event).

notification_routing_rule

Column	Type	Notes
id	TEXT PK	ULID
operator_code	TEXT NOT NULL	
event_type	TEXT NOT NULL	E.g., InvoiceIssued, PtpRegistered
channel	TEXT NOT NULL	EMAIL / SMS (initial scope); future channels added by adding adapter + adding enum value
template_purpose_code	TEXT NOT NULL	The purpose code that resolves to actual templates per format + locale at render time (see template table)
priority	INTEGER NOT NULL	Lower = tried first
urgency	TEXT NOT NULL	URGENT / NORMAL (urgent bypasses time-window restriction)
category	TEXT NOT NULL	TRANSACTIONAL / MARKETING (controls whether customer can opt out)
conditions	JSONB NULL	Optional payload-based conditions
enabled	BOOLEAN NOT NULL DEFAULT TRUE	Admin can pause
created_at	TIMESTAMPTZ NOT NULL	
updated_at	TIMESTAMPTZ NOT NULL	

Indexes: (operator_code, event_type, enabled, priority) for routing lookup.

Sample data:

operator_code	event_type	channel	priority	category	template_purpose_code
WIK	InvoiceIssued	EMAIL	1	TRANSACTIONAL	INVOICE_CYCLE_POST PAID
WIK	InvoiceIssued	SMS	2	TRANSACTIONAL	INVOICE_CYCLE_POST PAID
WIK	PtpRegistered	SMS	1	TRANSACTIONAL	PTP_CONFIRMATION
WIK	PtpRegistered	EMAIL	2	TRANSACTIONAL	PTP_CONFIRMATION
WIK	DisputeResolvedInFavor	EMAIL	1	TRANSACTIONAL	DISPUTE_RESOLVED_FAVOR

How to read: An InvoiceIssued event for WIK triggers an email (priority 1) and an SMS (priority 2). Both reference the same INVOICE_CYCLE_POSTPAID purpose code; NOT-01 resolves the actual templates per channel's format (EMAIL_SUBJECT + EMAIL_HTML + EMAIL_TEXT for the email; SMS_TEXT for the SMS) and per customer's locale.

channel_operator_config

Per-operator configuration for each enabled channel: which adapter to use, what credentials, what sender identifiers.

Column	Type	Notes
id	TEXT PK	ULID
operator_code	TEXT NOT NULL	
channel	TEXT NOT NULL	EMAIL / SMS (initial scope)
adapter_implementation	TEXT NOT NULL	Adapter class reference, e.g., smtp.default, sms.twilio, sms.africastalking. Allows multiple adapter implementations per channel.
sender_identifier	TEXT NOT NULL	Email: verified sender address (e.g., billing@wananchi.co.ke). SMS: registered sender ID / shortcode (e.g., WANANCHI, 21234).
credentials_ref	TEXT NOT NULL	Reference to secret in the secret manager (e.g., vault://notification/wik/email/smtp). Sensitive values never stored inline.
additional_config	JSONB NULL	Adapter-specific config (e.g., SMTP host/port, SMS aggregator endpoint, rate limits)
enabled	BOOLEAN NOT NULL DEFAULT TRUE	Admin can pause a channel for an operator

Column	Type	Notes
created_at	TIMESTAMPZ NOT NULL	
updated_at	TIMESTAMPZ NOT NULL	

Indexes: (operator_code, channel) UNIQUE, (operator_code, enabled).

Sample data:

operator_code	channel	adapter_implementation	sender_identifier	enabled
WIK	EMAIL	smtp.default	billing@wananchi.co.ke	true
WIK	SMS	sms.africastalking	WANANCHI	true
WUG	EMAIL	smtp.default	billing@wananchi.co.ug	true
WUG	SMS	sms.africastalking	WANANCHIUG	true
WTZ	EMAIL	smtp.default	billing@wananchi.co.tz	true
WTZ	SMS	sms.beemafrica	WANANCHI	false
YASSN	EMAIL	smtp.transac	facturation@yas.sn	true
YASSN	SMS	sms.orange-sn	YAS	true

How to read: Wananchi Kenya uses a default SMTP adapter for email with `billing@wananchi.co.ke` as sender, and the Africa's Talking SMS adapter with `WANANCHI` as sender ID. Wananchi Tanzania has SMS configured but currently disabled (the `enabled = false` row). Yas Senegal uses a different SMS adapter (`sms.orange-sn`) reflecting its different aggregator contract. The pluggable adapter design lets operators contract with different gateway providers without code changes — they just configure a different `adapter_implementation`.

customer_notification_preference

Column	Type	Notes
id	TEXT PK	ULID
customer_id	TEXT NOT NULL UNIQUE	
operator_code	TEXT NOT NULL	
email_opt_in	BOOLEAN NOT NULL DEFAULT TRUE	Marketing emails; transactional emails are always sent regardless
sms_opt_in	BOOLEAN NOT NULL DEFAULT TRUE	Marketing SMS; transactional SMS are always sent regardless
locale	TEXT NOT NULL	E.g., en, fr, sw
preferred_time_window_start	TIME NULL	E.g., 09:00. NULL = no preference
preferred_time_window_end	TIME NULL	E.g., 18:00
email_status	TEXT NOT NULL DEFAULT 'VALID'	VALID / SOFT_BOUNCED / INVALID
updated_at	TIMESTAMPZ NOT NULL	

Indexes: (customer_id) UNIQUE, (operator_code).

Sample data:

customer_id	operator	email_opt_in	sms_opt_in	locale	time_window	email_status
cust_01HX...A	WIK	true	true	en	NULL	VALID
cust_01HX...B	WIK	true	false	sw	NULL	VALID
cust_01HX...C	WIK	false	true	en	09:00-18:00	SOFT_BOUNCED
cust_01HX...D	WIK	true	true	en	NULL	INVALID
cust_01HX...E	YASSN	true	true	fr	NULL	VALID

How to read: Customer A receives everything by both channels in English. Customer B opted out of marketing SMS but still receives transactional SMS (invoices, payments) and all email. Customer C opted out of marketing email, only wants notifications between 09:00-18:00, and their email recently soft-bounced (NOT-01 will keep trying email but watch closely). Customer D's email is hard-bounced (INVALID) — email channel will be skipped entirely until an admin updates the email; SMS still works. Customer E is a Yas Senegal customer in French locale.

Future channels: when a new channel is added, this table gets a new opt-in column (e.g., `what sapp_opt_in`). Schema migration is additive; existing rows default to opt-out for the new channel.

notification_log

Aggregate row per notification event. Append-only.

Column	Type	Notes
id	TEXT PK	ULID — the notification_id
operator_code	TEXT NOT NULL	
customer_id	TEXT NOT NULL	
event_type	TEXT NOT NULL	Source event
source_event_id	TEXT NULL	Original event ID for traceability
source_entity_id	TEXT NULL	E.g., invoice_id
channels_attempted	JSONB NOT NULL	Array of channels actually tried
final_status	TEXT NOT NULL	DISPATCHED / PARTIALLY_DISPATCHED / SUPPRESSED / ESCALATED / UNDELIVERABLE
dispatched_at	TIMESTAMPZ NOT NULL	When processing completed (success or final failure)
manual_resend_by	TEXT NULL	Admin user ID if this is a manual resend
original_notification_id	TEXT NULL	If resend, points to the original

Indexes: (operator_code, customer_id, dispatched_at DESC), (event_type, dispatched_at DESC), (source_entity_id). P10Y retention.

Sample data:

id	event_type	source_entity_id	channels_attempted	final_status	dispatched_at	manual_resend_by
ntf_01HZ...A	InvoiceIssued	inv_01HZQ...	["EMAIL", "SMS"]	DISPATCHED	2026-05-01 00:16:05	NULL
ntf_01HZ...B	InvoiceIssued	inv_01HZR...	["EMAIL"]	DISPATCHED	2026-05-01 00:16:08	NULL
ntf_01HZ...C	PtpRegistered	ptp_01HZ...	["SMS", "EMAIL"]	PARTIALLY_DISPATCHED	2026-05-18 11:45:10	NULL
ntf_01HZ...D	InvoiceOverdue	inv_01HZS...	["EMAIL", "SMS"]	UNDELIVERABLE	2026-05-15 09:30:00	NULL
ntf_01HZ...E	InvoiceIssued	inv_01HZQ...	["EMAIL"]	DISPATCHED	2026-05-18 15:45:00	admin_user_42
ntf_01HZ...F	InvoiceIssued	inv_01HZT...	[]	SUPPRESSED	2026-05-18 12:00:00	NULL

How to read: Row A — a normal invoice issuance, both email and SMS sent successfully. Row B — same event for a different customer, only email sent (customer opted out of marketing SMS, though typically invoice notifications are transactional and not subject to opt-out; here this might be a marketing-category invoice summary). Row C — PTP confirmation tried both SMS (succeeded) and email (failed for some reason), hence PARTIALLY_DISPATCHED. Row D — overdue notification couldn't reach the customer on any channel (bad email, bad phone), marked UNDELIVERABLE for admin attention. Row E — the same invoice as row A, manually resent by admin user 42; original_notification_id would point to ntf_01HZ...A (omitted from this table for space). Row F — for some other invoice, no channels were attempted because the customer opted out of all applicable channels, so the notification was SUPPRESSED.

notification_delivery_attempt

Column	Type	Notes
id	TEXT PK	ULID
notification_id	TEXT NOT NULL	FK to notification_log
channel	TEXT NOT NULL	
template_id	TEXT NOT NULL	
recipient	TEXT NOT NULL	Email or phone (initial scope) — hashed if PII regulation requires
attempt_number	INTEGER NOT NULL	1, 2, 3 ...
status	TEXT NOT NULL	SENT / FAILED / PENDING_RETRY / ESCALATED
failure_category	TEXT NULL	TRANSIENT / PERMANENT_RECIPIENT / PERMANENT_TEMPLATE / PERMANENT_BUSINESS_RULE
failure_detail	TEXT NULL	Channel-specific error
channel_response	JSONB NULL	Raw response from channel (truncated if large)
attempted_at	TIMESTAMPZ NOT NULL	

Column	Type	Notes
next_attempt_at	TIMESTAMPZ NULL	If PENDING_RETRY

Indexes: (notification_id, attempt_number), (status, next_attempt_at) WHERE status = 'PENDING_RETRY' for retry scanner.

Sample data (showing several rows that together tell different stories):

id	notification_id	channel	attempt#	status	failure_category	attempted_at	next_attempt_at
att_01HZ...A1	ntf_01HZ...A	EMAIL	1	SENT	NULL	2026-05-01 00:16:03	NULL
att_01HZ...A2	ntf_01HZ...A	SMS	1	SENT	NULL	2026-05-01 00:16:05	NULL
att_01HZ...C1	ntf_01HZ...C	SMS	1	SENT	NULL	2026-05-18 11:45:02	NULL
att_01HZ...C2	ntf_01HZ...C	EMAIL	1	FAILED	TRANSIENT	2026-05-18 11:45:03	2026-05-18 11:46:03
att_01HZ...C3	ntf_01HZ...C	EMAIL	2	FAILED	TRANSIENT	2026-05-18 11:46:08	2026-05-18 11:51:08
att_01HZ...C4	ntf_01HZ...C	EMAIL	3	SENT	NULL	2026-05-18 11:51:15	NULL
att_01HZ...D1	ntf_01HZ...D	EMAIL	1	FAILED	PERMANENT_RECIPIENT	2026-05-15 09:30:01	NULL
att_01HZ...D2	ntf_01HZ...D	SMS	1	FAILED	PERMANENT_RECIPIENT	2026-05-15 09:30:03	NULL
att_01HZ...G1	ntf_01HZ...G	EMAIL	1	FAILED	TRANSIENT	2026-05-18 13:00:00	2026-05-18 13:01:00
att_01HZ...G2-7	ntf_01HZ...G	EMAIL	2-7	FAILED	TRANSIENT	(six more rows over 8 hours)	(each scheduled per backoff)
att_01HZ...G8	ntf_01HZ...G	EMAIL	8	ESCALATED	TRANSIENT	2026-05-18 21:00:00	NULL

How to read: Rows A1/A2 — single successful attempts for both email and SMS, the simple happy path. Rows C1-C4 tell a retry story: SMS succeeded on first try; email had a transient failure (gateway timeout maybe), got retried at 1 minute, failed again, retried at 5 minutes, succeeded — total 3 email attempts, eventual success contributing to the parent notification's PARTIALLY_DISPATCHED because by the time email finally went through, the SMS had already been counted (or vice versa, depending on aggregation logic). Rows D1/D2 — both channels failed with PERMANENT_RECIPIENT (invalid email AND invalid phone), so no retries scheduled, no fallback worked — parent notification is UNDELIVERABLE. Rows G1-G8 — a notification that hit transient failures 8 times in a row over ~8 hours of exponential backoff, finally ESCALATED to ops review after retry limit; no further automatic attempts.

render_failure_queue

Column	Type	Notes
id	TEXT PK	ULID
event_type	TEXT NOT NULL	
source_entity_id	TEXT NOT NULL	
operator_code	TEXT NOT NULL	
failure_reason	TEXT NOT NULL	E.g., TEMPLATE_NOT_FOUND, RENDER_ENGINE_ERROR, MALFORMED_PAYLOAD
failure_detail	TEXT NULL	
attempt_count	INTEGER NOT NULL DEFAULT 1	
last_attempt_at	TIMESTAMPZ NOT NULL	
next_attempt_at	TIMESTAMPZ NULL	If auto-retry pending
status	TEXT NOT NULL	PENDING_RETRY / GAVE_UP_AUTO / RESOLVED
original_event_payload	JSONB NOT NULL	For re-render
resolved_by	TEXT NULL	Admin user ID
resolved_at	TIMESTAMPZ NULL	

Indexes: (status, next_attempt_at) for scanner; (operator_code, status) for admin queue view.

Sample data:

id	event_type	source_entity_id	operator	failure_reason	attempt_count	status	resolved_by
rfq_01HZ...A	InvoiceIssued	inv_01HZ...	WIK	TEMPLATE_NOT_FOUND	3	PENDING_RETRY	NULL
rfq_01HZ...B	CreditNoteIssued	inv_01HZ...	WIK	RENDER_ENGINE_ERROR	8	GAVE_UP_AUTO	NULL
rfq_01HZ...C	TaxInvoiceSigned	inv_01HZ...	YASSN	MALFORMED_PAYLOAD	1	RESOLVED	admin_user_17
rfq_01HZ...D	DunningStepTriggered	inv_01HZ...	WUG	TEMPLATE_NOT_FOUND	8	GAVE_UP_AUTO	NULL

How to read: Row A — a recent template-not-found failure being retried (likely the operator's PDF template upload was forgotten or the engine couldn't locate the right locale version). After 8 retries it'll move to GAVE_UP_AUTO. Row B — a credit note render that hit a render-engine error and reached the auto-retry limit (GAVE_UP_AUTO); admin needs to look at it. Row C — a tax invoice render that failed once due to malformed payload, but an admin already fixed it manually (RESOLVED with resolved_by populated; probably triggered a re-render via the admin endpoint that succeeded). Row D — a dunning letter for Wananchi Uganda gave up after 8 attempts because the template was never created for that operator + step; sits in admin's queue.

Channel adapter contract

The ChannelAdapter interface is the contract that makes NOT-01 extensible. Adding a new channel means writing one new adapter; nothing else in NOT-01 changes. This section documents the contract precisely enough that a developer can implement a new adapter from this DD alone.

Java interface

```
package com.sophix.notification.dispatch.adapter;

import java.time.Duration;
import java.util.Map;
import java.util.Optional;

/**
 * Channel adapter contract. One implementation per channel (email, SMS, future channels).
 *
 * Lifecycle: instantiated once per (channel, operator_code) pair at application startup,
 * cached by the dispatcher, and reused for all dispatches on that channel + operator.
 * Adapter instances are SHARED across threads and MUST be thread-safe.
 *
 * The adapter does NOT decide who to send to, what template to use, or whether to send –
 * those decisions are owned by the routing module. The adapter receives a fully-resolved
 * Dispatch and produces a DeliveryResult.
 */
public interface ChannelAdapter {

    /**
     * Channel this adapter handles. Used by the dispatcher to route Dispatch instances
     * to the correct adapter implementation.
     */
    Channel channel();

    /**
     * Called once when the adapter instance is created. Receives the per-operator config
     * row for this (channel, operator_code) and the resolved credentials from the secret
     * store. The adapter validates the config and prepares any reusable resources
     * (SMTP connection pool, HTTP client, etc.).
     *
     * Throws AdapterConfigurationException if the config is invalid or required credentials
     * are missing – caught by the dispatcher, which marks the adapter UNUSABLE and emits
     * an operational alert. The dispatcher does NOT retry initialization automatically.
     *
     * @param config the row from channel_operator_config + decrypted credentials
     */
    void initialize(ChannelOperatorConfig config) throws AdapterConfigurationException;

    /**
     * Send one dispatch through the channel. MUST be idempotent on the adapter's side:
     * if the dispatcher retries the same Dispatch (same dispatch_id), the adapter MUST
     * either (a) re-send accepting that the recipient may receive a duplicate, or
     * (b) detect the duplicate via the channel's deduplication mechanism if available
     * and return a SENT result without re-sending. Most channels don't have native
     * deduplication, so (a) is the default – the dispatcher's idempotency layer (Redis
     * key per source event) prevents most duplicates at a higher level.
     */
}
```

```

    *
    * MUST respect the timeout parameter – return TRANSIENT failure if the channel
    * doesn't respond within the timeout rather than blocking indefinitely.
    *
    * MUST NOT mutate any shared state visible outside the adapter.
    * MUST NOT log PII at INFO level (recipient address / phone are PII).
    *
    * @param dispatch the resolved dispatch – rendered artifacts, recipient, metadata
    * @param timeout max time to wait for channel response
    * @return result with status (SENT or FAILED), failure category if failed, and
    *         channel-specific response data for audit
    */
    DeliveryResult send(Dispatch dispatch, Duration timeout);

    /**
     * Called once when the application is shutting down. Releases resources (connection
     * pools, file handles). MUST complete within 10 seconds; otherwise the JVM terminates
     * forcibly.
     */
    void shutdown();
}

```

Supporting types

```

/**
 * Channel enum. New channels add entries here AND register a corresponding adapter
 * implementation. Initial scope: EMAIL, SMS.
 */
public enum Channel {
    EMAIL,
    SMS
    // Future: WHATSAPP, TELEGRAM, PUSH, VOICE, PAPER_MAIL, ...
}

/**
 * Per-operator channel configuration passed to initialize().
 */
public final class ChannelOperatorConfig {
    private final String operatorCode;
    private final Channel channel;
    private final String senderIdentifier; // email "From:" address, SMS sender ID
    private final Map<String, String> credentials; // resolved from secret store
    private final Map<String, Object> additionalConfig; // free-form per-adapter
    // getters omitted
}

/**
 * One dispatch – a resolved instruction "send this content to this recipient through
 * this channel." All template resolution and rendering already done by upstream
 * (TemplateLookupService + TemplateEngine). The adapter just transmits.
 */
public final class Dispatch {
    private final String dispatchId; // ULID, used for idempotency
    private final String notificationId; // ULID, FK to notification_log
    private final String operatorCode;
    private final String customerId;
    private final String recipient; // email address or phone number
    private final Map<TemplateFormat, String> renderedArtifacts;
    // e.g., EMAIL_SUBJECT -> "Your invoice...",
    // EMAIL_HTML -> "<html>...",
    // EMAIL_TEXT -> "Your invoice...",
    // SMS_TEXT -> "Invoice INV-..."
    private final Optional<String> pdfStorageKey; // MinIO key if a PDF should be attached
    // or linked; adapter decides which
    private final Map<String, Object> metadata; // tracking info for the audit row
    // getters omitted
}

/**
 * Result returned by send().
 */
public final class DeliveryResult {
    private final DeliveryStatus status; // SENT or FAILED
    private final Optional<FailureCategory> category; // populated when status = FAILED
    private final Optional<String> failureDetail; // channel-specific error message
    private final Optional<String> externalReference; // channel-generated ID for tracking
    // (e.g., SMTP message-id, SMS DLR ref)
    private final Map<String, Object> channelResponse; // raw response for audit
    // (truncated if > 4KB)
    // getters omitted

    public static DeliveryResult sent(String externalReference,
                                     Map<String, Object> channelResponse) { ... }
}

```

```

        public static DeliveryResult failed(FailureCategory category,
                                           String detail,
                                           Map<String, Object> channelResponse) { ... }
    }

    public enum DeliveryStatus { SENT, FAILED }

    public enum FailureCategory {
        TRANSIENT,           // gateway timeout, 5xx, rate limit – retry sensible
        PERMANENT_RECIPIENT,  // invalid email/phone – won't succeed without contact update
        PERMANENT_TEMPLATE,   // channel rejected content – needs template fix
        PERMANENT_BUSINESS_RULE // recipient blocked at carrier level, blocked region
    }

    public final class AdapterConfigurationException extends Exception { ... }

```

Contract rules

A new adapter implementation **MUST** satisfy:

Rule	What it means
Thread-safe	<code>send()</code> is called from multiple threads concurrently. The adapter must not have shared mutable state without synchronization. Connection pools, HTTP clients, and similar resources are fine — they're designed for concurrent use.
No side effects beyond the channel call	The adapter MUST NOT write to the database, emit Kafka events, update caches, or modify any state outside its own internal resources. The dispatcher owns audit and event emission.
Categorize failures correctly	A TRANSIENT failure incorrectly marked as PERMANENT means the customer never receives the message even though a retry would have worked. A PERMANENT failure marked TRANSIENT means 8 hours of pointless retries. Each adapter MUST document its category mapping from the channel's error codes in a comment block.
Respect the timeout	If the channel doesn't respond within <code>timeout</code> , return TRANSIENT failure with detail "timeout after Xs". Do not block indefinitely; that blocks the dispatcher thread.
No PII in logs at INFO	Recipient address, phone number, and message content are PII. At INFO level, log only <code>dispatch_id</code> , <code>notification_id</code> , <code>status</code> , <code>category</code> . PII may appear at DEBUG (operator-controlled).
Idempotent at the dispatcher contract	The dispatcher may call <code>send()</code> more than once with the same <code>dispatch_id</code> (during retry, after restart). The adapter sends each time unless the channel itself supports deduplication. The dispatcher's higher-level idempotency (Redis key per source event) prevents most duplicates; the adapter doesn't need its own dedup layer.
Capture externalReference	If the channel returns a message ID, tracking ID, or similar reference, populate <code>externalReference</code> in the result. This enables later DLR (delivery receipt) correlation and customer-support troubleshooting.
Truncate channelResponse to 4KB	Some channel responses are large (full HTTP response bodies). Adapters MUST truncate to 4KB before returning so the audit table doesn't bloat.

Worked example: EmailAdapter skeleton

```

package com.sophix.notification.dispatch.adapter;

import jakarta.mail.*;
import jakarta.mail.internet.*;
import java.time.Duration;
import java.util.Properties;

public final class EmailAdapter implements ChannelAdapter {

    private Session smtpSession;
    private String senderAddress;
    private MinioClient minioClient; // injected for PDF attachment fetch
    private boolean attachPdf;       // operator config: attach vs link

    @Override
    public Channel channel() {
        return Channel.EMAIL;
    }

    @Override
    public void initialize(ChannelOperatorConfig config) throws AdapterConfigurationException {
        Map<String, String> creds = config.getCredentials();
        if (!creds.containsKey("smtp_host") || !creds.containsKey("smtp_user")) {
            throw new AdapterConfigurationException(
                "EmailAdapter requires 'smtp_host' and 'smtp_user' credentials"
            );
        }
    }
}

```

```

    });
}

Properties props = new Properties();
props.put("mail.smtp.host", creds.get("smtp_host"));
props.put("mail.smtp.port", creds.getOrDefault("smtp_port", "587"));
props.put("mail.smtp.starttls.enable", "true");
props.put("mail.smtp.auth", "true");

this.smtpSession = Session.getInstance(props, new Authenticator() {
    @Override protected PasswordAuthentication getPasswordAuthentication() {
        return new PasswordAuthentication(
            creds.get("smtp_user"),
            creds.get("smtp_password")
        );
    }
});

this.senderAddress = config.getSenderIdentifier();
this.attachPdf = Boolean.parseBoolean(
    config.getAdditionalConfig().getOrDefault("attach_pdf", "true").toString()
);
}

@Override
public DeliveryResult send(Dispatch dispatch, Duration timeout) {
    try {
        MimeMessage msg = new MimeMessage(smtpSession);
        msg.setFrom(new InternetAddress(senderAddress));
        msg.addRecipient(Message.RecipientType.TO, new InternetAddress(dispatch.getRecipient()));
        msg.setSubject(dispatch.getRenderedArtifacts().get(TemplateFormat.EMAIL_SUBJECT));

        Multipart multipart = new MimeMultipart("alternative");

        // Text part
        MimeBodyPart textPart = new MimeBodyPart();
        textPart.setText(
            dispatch.getRenderedArtifacts().get(TemplateFormat.EMAIL_TEXT),
            "UTF-8"
        );
        multipart.addBodyPart(textPart);

        // HTML part
        MimeBodyPart htmlPart = new MimeBodyPart();
        htmlPart.setContent(
            dispatch.getRenderedArtifacts().get(TemplateFormat.EMAIL_HTML),
            "text/html; charset=UTF-8"
        );
        multipart.addBodyPart(htmlPart);

        // PDF attachment (if configured and present)
        if (attachPdf && dispatch.getPdfStorageKey().isPresent()) {
            MimeBodyPart pdfPart = new MimeBodyPart();
            byte[] pdfBytes = minioClient.getObject(
                dispatch.getOperatorCode(),
                dispatch.getPdfStorageKey().get()
            );
            pdfPart.setContent(pdfBytes, "application/pdf");
            pdfPart.setFileName(dispatch.getPdfStorageKey().get().split("/")[3]);
            multipart.addBodyPart(pdfPart);
        }

        msg.setContent(multipart);

        // Send with timeout (SMTP transport blocks; would wrap in executor in practice)
        Transport.send(msg);

        return DeliveryResult.sent(
            msg.getMessageID(),
            Map.of("smtp_message_id", msg.getMessageID())
        );
    } catch (SendFailedException e) {
        return DeliveryResult.failed(
            categorize(e),
            e.getMessage(),
            Map.of("error_class", e.getClass().getSimpleName())
        );
    } catch (MessagingException e) {
        return DeliveryResult.failed(
            FailureCategory.TRANSIENT,
            e.getMessage(),

```

```

        Map.of("error_class", e.getClass().getSimpleName())
    );
}
}

/**
 * Category mapping for SMTP errors.
 *
 * SMTP 5xx codes (permanent):
 * 550, 553 – invalid recipient          -> PERMANENT_RECIPIENT
 * 552      – message size exceeds limit  -> PERMANENT_TEMPLATE
 * 554      – transaction failed (often spam-flag) -> PERMANENT_TEMPLATE
 *
 * SMTP 4xx codes (transient):
 * 421 – service not available          -> TRANSIENT
 * 450 – mailbox unavailable             -> TRANSIENT
 * 451 – local error                    -> TRANSIENT
 * 452 – insufficient storage            -> TRANSIENT
 *
 * Connection-level errors (timeout, refused) -> TRANSIENT
 */
private FailureCategory categorize(SendFailedException e) {
    int code = parseSmtpCode(e);
    if (code >= 500) {
        if (code == 550 || code == 553) return FailureCategory.PERMANENT_RECIPIENT;
        return FailureCategory.PERMANENT_TEMPLATE;
    }
    return FailureCategory.TRANSIENT;
}

private int parseSmtpCode(SendFailedException e) {
    // implementation extracts the SMTP code from the exception message
    return 0;
}

@Override
public void shutdown() {
    // jakarta.mail Session has no explicit close; SMTP connections are per-send
}
}

```

Worked example: SmsAdapter skeleton (shape only)

```

public final class SmsAdapter implements ChannelAdapter {

    private SmsGatewayClient client; // adapter-specific HTTP client
    private String senderId;
    private int maxSmsParts; // operator config: max concatenated parts

    @Override public Channel channel() { return Channel.SMS; }

    @Override
    public void initialize(ChannelOperatorConfig config) throws AdapterConfigurationException {
        // similar to EmailAdapter: validate credentials, instantiate the gateway client
        // ...
    }

    @Override
    public DeliveryResult send(Dispatch dispatch, Duration timeout) {
        String text = dispatch.getRenderedArtifacts().get(TemplateFormat.SMS_TEXT);

        // Truncate or split into parts as configured
        if (text.length() > 160 * maxSmsParts) {
            text = text.substring(0, 160 * maxSmsParts - 3) + "...";
        }

        try {
            SmsResponse response = client.send(
                senderId,
                dispatch.getRecipient(),
                text,
                timeout
            );
            return DeliveryResult.sent(
                response.getMessageId(),
                Map.of(
                    "gateway_message_id", response.getMessageId(),
                    "parts", response.getParts(),
                    "cost", response.getCost()
                )
            );
        } catch (SmsGatewayException e) {
            return DeliveryResult.failed(

```

```

        categorize(e),
        e.getMessage(),
        Map.of("error_code", e.getErrorCode())
    );
}

}

/**
 * Category mapping for SMS gateway errors. Each gateway has its own code system;
 * this adapter is gateway-specific (different SmsAdapter for Africa's Talking vs
 * Beem Africa vs Orange-SN). Operators select the adapter via `adapter_implementation`
 * in channel_operator_config.
 */
private FailureCategory categorize(SmsGatewayException e) {
    // gateway-specific mapping
    return FailureCategory.TRANSIENT;
}

@Override
public void shutdown() {
    client.close();
}
}

```

Adding a new channel — checklist for the implementer

When adding a new channel (e.g., WhatsApp) later:

1. Add the enum value to `Channel` (e.g., `WHATSAPP`)
2. Add template format(s) to `TemplateFormat` if needed (e.g., `WHATSAPP_TEMPLATE_REF`)
3. Implement a new `ChannelAdapter` for the channel
4. Register the adapter implementation in the dispatcher's adapter registry
5. Add a column to `customer_notification_preference` for the new opt-in (additive migration)
6. Add per-operator rows to `channel_operator_config` for operators that enable the channel
7. Operators add routing rules that reference the new channel
8. Document the channel's adapter implementation reference in the channel-specific notes section

Nothing else in NOT-01 changes. The routing engine, the preferences engine, the failure handling, the retry scheduler, the audit tables, the admin operations — all unaffected.

API surface

Mechanism	Used for	Examples
Kafka consumption	Primary input — events from all source DDs	<code>InvoiceIssued</code> , <code>CreditNoteIssued</code> , <code>TaxInvoiceSigned</code> , <code>PtpRegistered</code> , <code>DisputeOpened</code> , <code>InvoiceOverdue</code> , <code>DunningStepTriggered</code> , etc.
Kafka emission	Status updates back to source DDs and analytics	<code>PdfReady</code> , <code>RenderFailed</code> , <code>NotificationDispatched</code> , <code>NotificationFailed</code> , <code>NotificationSuppressed</code> , <code>BounceProcessed</code>
REST (admin-facing)	Template management, manual resend, dashboard, failure recovery	<code>POST /api/admin/templates</code> , <code>POST /api/admin/notifications/{id}/resend</code> , <code>GET /api/admin/notifications/dashboard</code>
REST (customer-facing)	Self-service preference management	<code>GET /api/notifications/preferences</code> , <code>PUT /api/notifications/preferences</code>
MinIO read/write	PDF storage	Per <code>FOUNDATION_FILE_STORAGE.md</code>
External channel APIs	Outbound: SMTP for email; SMSC API / aggregator for SMS (initial scope). Future channels add their own external APIs.	Per-operator credentials in <code>channel_operator_config</code>

Request / response samples

Customer reads their preferences

```

GET /api/notifications/preferences HTTP/1.1
Authorization: Bearer <customer-JWT>

{
  "customer_id": "cust_01HX...",

```

```

    "operator_code": "WIK",
    "email_opt_in": true,
    "sms_opt_in": true,
    "locale": "en",
    "preferred_time_window_start": null,
    "preferred_time_window_end": null,
    "email_status": "VALID"
  }
}

```

Customer updates preferences (opts out of SMS marketing)

```

PUT /api/notifications/preferences HTTP/1.1
Authorization: Bearer <customer-JWT>
Content-Type: application/json

```

```

{
  "sms_opt_in": false,
  "locale": "fr"
}

{
  "message": "Preferences updated. SMS marketing will no longer be sent. Transactional SMS (invoices, payments, security)",
  "updated_at": "2026-05-18T15:30:00Z"
}

```

Admin manual resend

```

POST /api/admin/notifications/ntf_01HZ.../resend HTTP/1.1
Authorization: Bearer <admin-JWT>
Content-Type: application/json

```

```

{
  "channels": ["EMAIL"],
  "reason": "Customer reported they did not receive the original invoice email."
}

{
  "new_notification_id": "ntf_01HZ_NEW",
  "original_notification_id": "ntf_01HZ...",
  "channels_attempted": ["EMAIL"],
  "final_status": "DISPATCHED",
  "dispatched_at": "2026-05-18T15:45:00Z"
}

```

Admin uploads new template version

```

POST /api/admin/templates HTTP/1.1
Authorization: Bearer <admin-JWT>
Content-Type: multipart/form-data

```

```

operator_code=WIK
template_format=PDF
template_purpose_code=INVOICE_CYCLE_POSTPAID
locale=en
engine_type=HTML_TO_PDF
template_file=<binary>
sample_data={"invoice_id":"inv_01HZ_SAMPLE","customer":...}

{
  "template_id": "tpl_01HZ_NEW",
  "version": 4,
  "status": "DRAFT",
  "preview_url": "/api/admin/templates/tpl_01HZ_NEW/preview"
}

```

Cross-DD coordination

Touched	What NOT-01 owns	What the other DD owns
BIL-02 Core	Reads customer_snapshot from invoice/credit note/etc. rows when rendering; writes metadata.pdf_url back.	Owens the entity.
BIL-02-GEN-01	Consumes InvoiceIssued, CreditNoteIssued, DebitNoteIssued.	Generation.
BIL-02-TAX-01	Consumes TaxInvoiceSigned (NOT TaxInvoiceIssued — renders only after gateway signing).	Tax invoice lifecycle.
BIL-02-ADJ-01	Receives credit/debit note events (via GEN-01) for rendering and notification.	Adjustment workflow.

Touched	What NOT-01 owns	What the other DD owns
BIL-02-READ-01	Emits PdfReady consumed by READ-01 for cache invalidation. Consumes PdfRerenderRequested from READ-01 admin operations.	Read API.
BIL-02-STA-01	Consumes status / PTP / dispute events for customer notifications (confirmations, acknowledgments).	Status management.
BIL-01 Charging Engine	Consumes PaymentApplied (payment receipt notification), PaymentFailed.	Payment flow.
BIL-04 Collections	Consumes DunningStepTriggered — renders dunning letter PDF, dispatches via configured channels per step.	Dunning policy.
SUB-* Subscription	Consumes subscription lifecycle events (welcome, change, suspension) — TBD as SUB DDs are written.	Subscription.
CRM	Reads customer master (email, phone, language) via service call.	Customer identity.
MinIO	Reads/writes PDFs in invoices, dunning buckets per FOUNDATION_FILE_STORAGE.md.	Storage infrastructure.
External: SMTP servers	Per-operator email delivery.	Operator infra / third-party transactional email service.
External: SMS gateways	Per-operator SMS delivery.	Operator-contracted SMS aggregator.

Dependencies

Dependency	Owner
All source DDs that emit events NOT-01 consumes	Various teams
FOUNDATION_FILE_STORAGE.md (MinIO)	infrastructure team
FOUNDATION_KAFKA.md	infrastructure team
FOUNDATION_DB.md (cluster N)	infrastructure team
FOUNDATION_AUTH.md (NOTIFICATION_SENDER, TEMPLATE_MANAGER roles)	infrastructure team
Templating engine (deployment choice — Handlebars, Mustache, Thymeleaf, HTML-to-PDF tool, etc.)	infrastructure team / NOT-01 team
Per-operator SMTP server / email service contract	each operator
Per-operator SMS gateway contract	each operator
Per-operator document templates	each operator's billing/marketing team
Per-operator message templates	each operator's communications team
Per-operator routing rules	each operator's communications team

Operator-configurable capabilities

Capability	Where configured	Notes
Channel enablement	channel_operator_config	Each operator enables/disables specific channels (initial scope: EMAIL, SMS — both typically enabled)
Channel credentials	channel_operator_config + secret store	SMTP credentials, SMS API keys
Templates	template table	Per (operator, template_format, template_purpose_code, locale, version). Operator commits its own templates
Routing rules	notification_routing_rule	Event → channel + template_purpose_code mappings
Default customer preferences	Per-operator config	E.g., default sms_opt_in TRUE in market A, FALSE in market B
Regulatory time-window restrictions	Per-operator config	E.g., no SMS 21:00-07:00 in market A
Pro forma rendering	Per-operator config	Render or skip
Render retry schedule	Per-operator config	Default 8 retries; can be customized
Email sender identity	Per-operator config	Verified sender per operator

Capability	Where configured	Notes
SMS sender ID / shortcode	Per-operator config	Operator's registered SMS sender
Time zone	Per-operator config	For time-window evaluation

B — Target operator coverage

Each operator deploys their own per-channel configuration, templates, and routing rules. The four target operators (Wananchi KE, Wananchi UG, Wananchi TZ, Yas SN) will each set up:

- SMTP / email service contract → EmailAdapter config
- SMS gateway / aggregator → SmsAdapter config
- Per-operator templates per (`template_format`, `template_purpose_code`, `locale`)
- Per-operator routing rule set

When additional channels are added in future phases (e.g., WhatsApp, in-app push), each operator that wants them sets up the corresponding external service contract and adapter config; existing email/SMS deployments are unaffected.

The DD describes the platform capability; per-operator setup is per-operator implementation work outside this DD.

C — Current TZ and gap

Workshop materials do not describe a unified notification service in TZ Sophix. Various per-DD notification mechanisms likely exist (email triggers in billing, SMS triggers in subscription, etc.) but there is no documented service to compare against.

NOT-01 is entirely new functionality as a unified service. It introduces:

- Single notification surface across channels (initial scope: email, SMS)
- Channel-agnostic adapter contract enabling future channel additions without changing routing or preferences
- Per-operator template management with versioning, format-agnostic (PDF, email parts, SMS text — same template model)
- Per-customer preference management
- Centralized routing rules
- Comprehensive audit
- Artifact rendering separated from event sources
- Failure handling with retries and channel fallback